

601.445/645

Practical Cryptographic Systems

Symmetric Cryptography IV

Instructor: Matthew Green

Housekeeping

- **Quizzes graded**
 - Reading: sorry, it is up now
 - For your Agent reports:
- **The roster.json file is up (in the Git repository), and the submission box is up**

News

News

September 15, 2026

Apple Reference Image: A New Approach for Verified Photography

Written by Apple Security Engineering and Architecture (SEAR) and Camera & Photos



Today, powerful, widely available AI tools allow users to easily generate or alter photorealistic images to a degree that was difficult to imagine just a few years ago. These tools enable helpful features, like one-touch removal of background distractions, but they also make it difficult to distinguish between photographs that depict real events, and synthetic images that are heavily altered or entirely generated. So, in the case where the essential role of a photograph is to prove that something actually happened, an image appearing photorealistic is no longer sufficient to establish its veracity.



Cancel

Allow

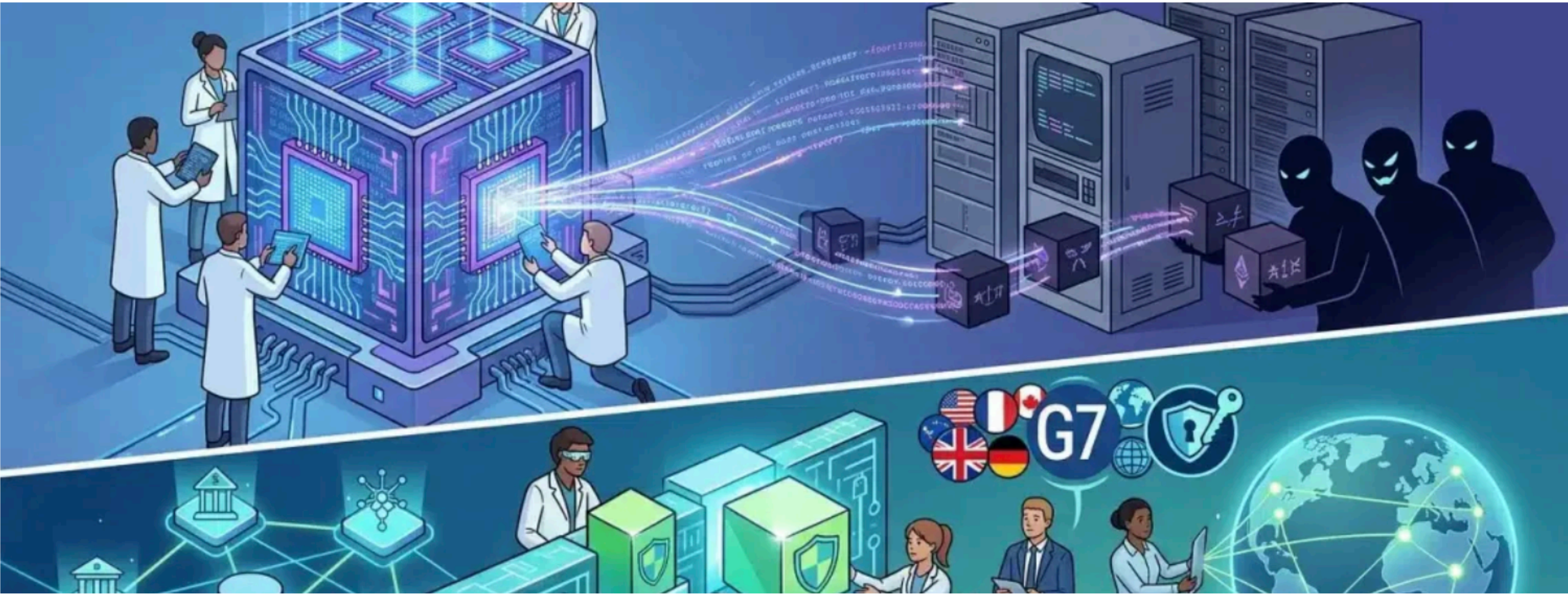
Home > Firewall Daily

G7, CISA Urge Urgent Shift to Post-Quantum Cryptography



by Samiksha Jain — September 7, 2026 in Firewall Daily, Cyber News

🔖 0



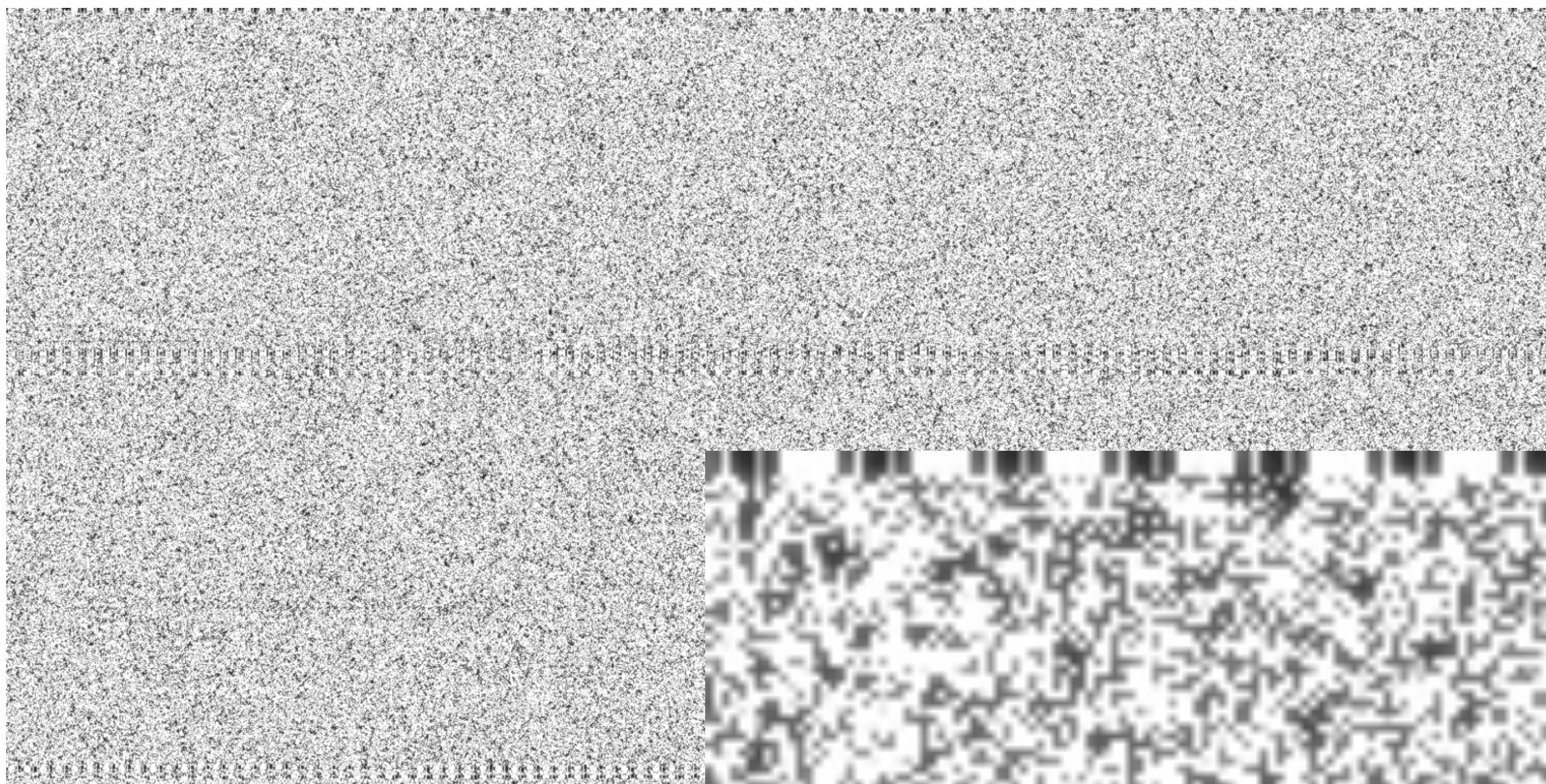
Review

- Last time:
 - We talked about modes of operation
 - ECB mode
 - Semantic security (IND-CPA)

Review

- ECB Mode: Encrypt each block separately
- Problems?





Security of Encryption

- Semantic Security
 - Due to Goldwasser & Micali (1980s)
 - Informally: An encryption scheme is secure if adversary who sees ciphertext “learns as much” as adversary who doesn’t see ciphertext.
- Even if adversary can request chosen plaintexts
 - How do we state this formally?

ECB Mode

- ECB is deterministic
- Leads to problems, e.g.,:



Review

- Semantic Security (IND-CPA)



Adversary

For $i=1$ to q :

M_i

$E(k, M_i)$

M^*_0, M^*_1

$C^* = E(M_b)$

b'

$b \xleftarrow{\$} \{0, 1\}$



k

Adversary “wins” if $b=b'$

Review

A's "advantage" $\text{Adv}^{\text{IND-CPA}}(A) = \Pr[A \text{ wins}] - 1/2$

A scheme is IND-CPA secure if for all p.p.t. adversaries A ,

$\text{Adv}^{\text{IND-CPA}}(A)$ is negligible

- Semantic Security (IND-CPA)



Adversary

For $i=1$ to q :

M_i

$E(k, M_i)$

M^*_0, M^*_1

$C^* = E(M_b)$

b'

$b \xleftarrow{\$} \{0, 1\}$



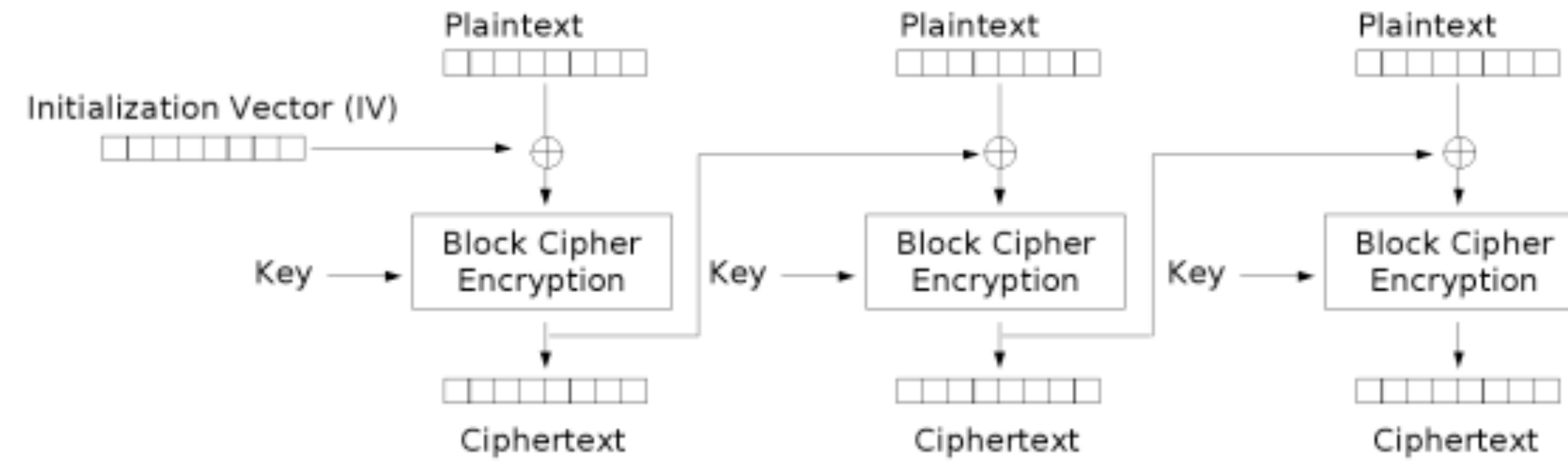
k

Adversary "wins" if $b == b'$

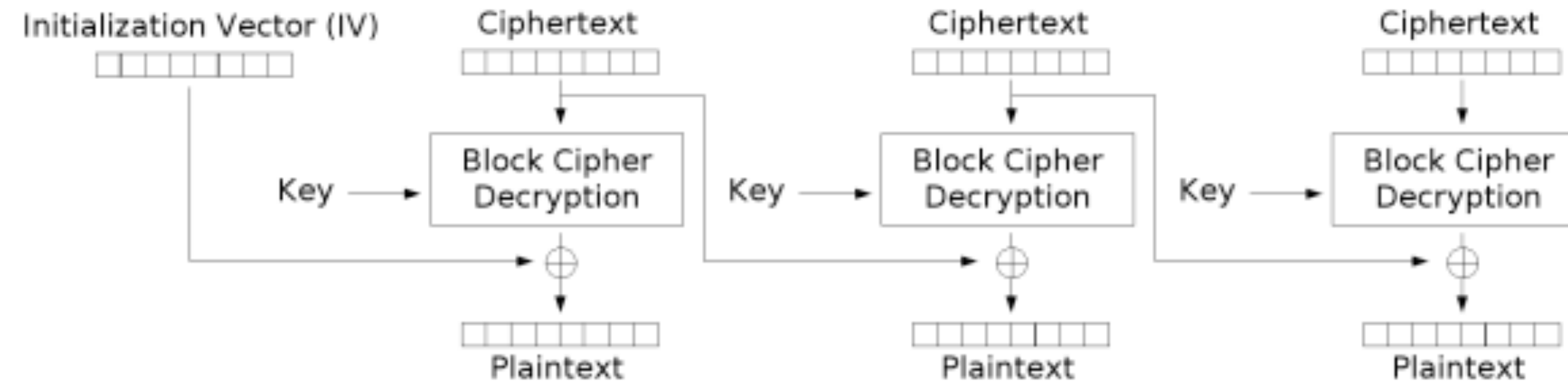
Using Block Ciphers

- ECB is not semantically secure, hence we use a “mode of operation”
 - e.g., CBC, CTR, CFB, OFB (and others)
- These provide:
 - Security for multi-block messages
 - Randomization (through an Initialization Vector)

CBC Mode



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

What if the message is not block aligned?

Security of CBC

- Is CBC a secure encryption scheme?
 - Yes, assuming a secure block cipher
 - Correct (random) IV generation
 - Can prove this under assumption that block cipher = Pseudo-Random Permutation (PRP)
- Bellare, Desai, Jorikpää & Rogaway (2000)
 - Easy to use wrong...
 - Most important: use a unique & random IV!

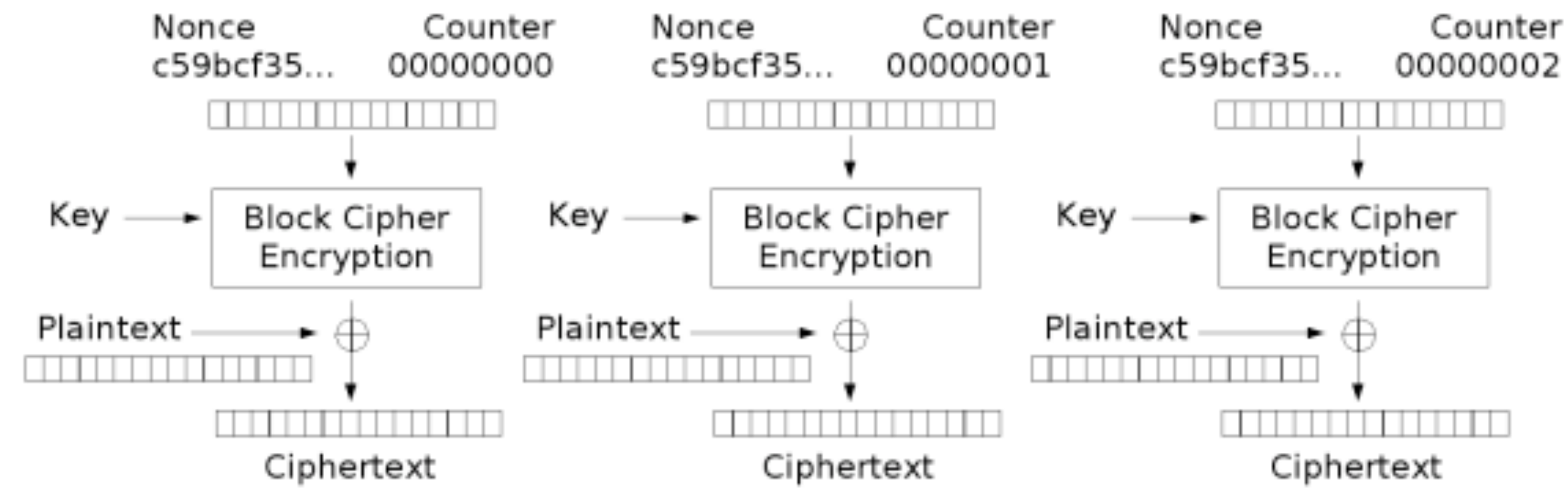
The size of the frame of data to be encrypted or decrypted (i.e. how often a new CBC chain is started) depends on the particular application, and is defined for each in the corresponding format specific books of this specification. Unless otherwise specified, the Initialization Vector used at the beginning of a CBC encryption or decryption chain is a constant, iv_0 , which is:

0BA0F8DDFEA61FB3D8DF9F566A050F78₁₆

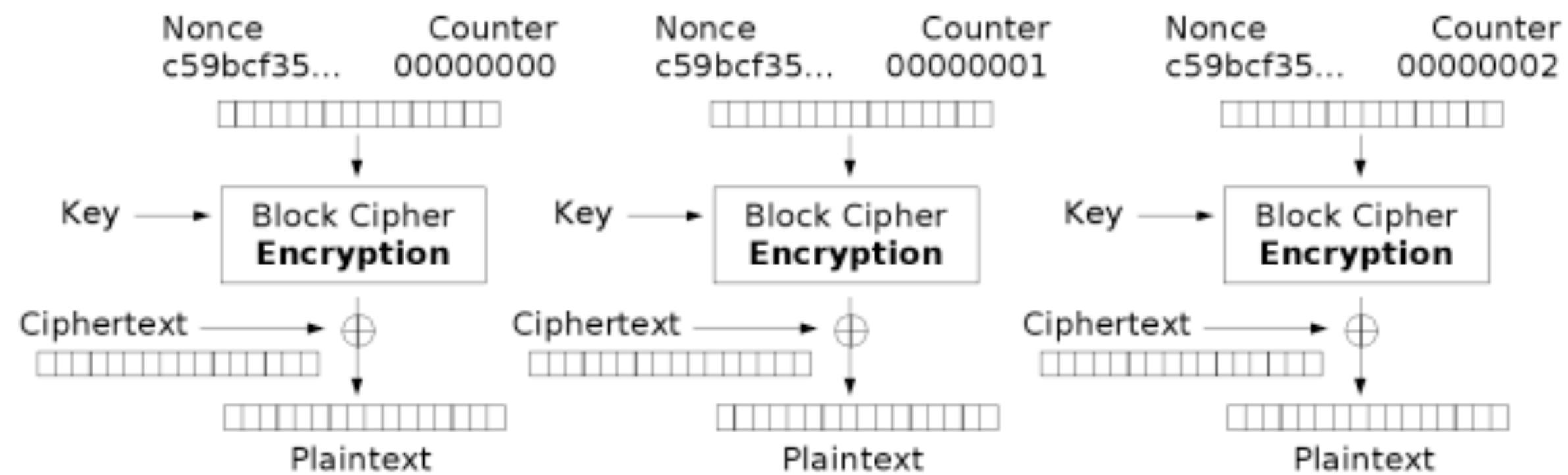
Advanced Access Content System (AACCS)

Introduction and Common Cryptographic Elements

CTR Mode



Counter (CTR) mode encryption



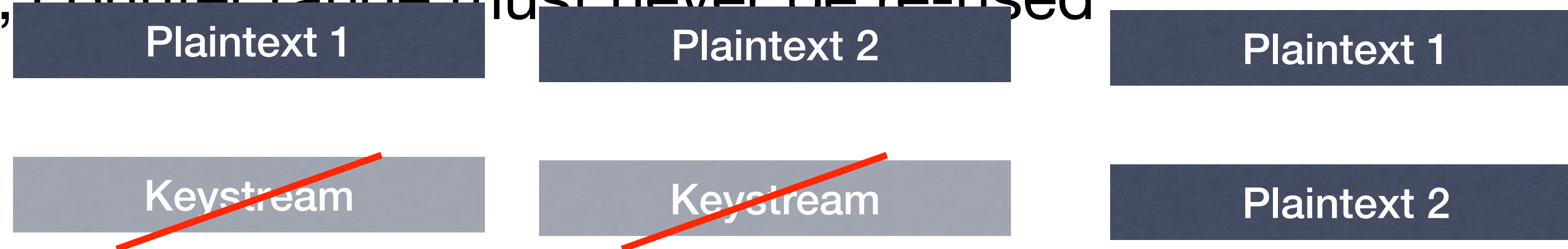
Counter (CTR) mode decryption

CTR (intuition)

- CTR uses the cipher to expand a short cipher key (K) into a long string of pseudorandom output bits
- This is called a “**keystream**”
- We then XOR the keystream with a long message (or many messages)
- This turns a block cipher into a “stream cipher”

Security of CTR

- Yes, assuming secure block cipher (PRP)
- However, counter range must never be re-used



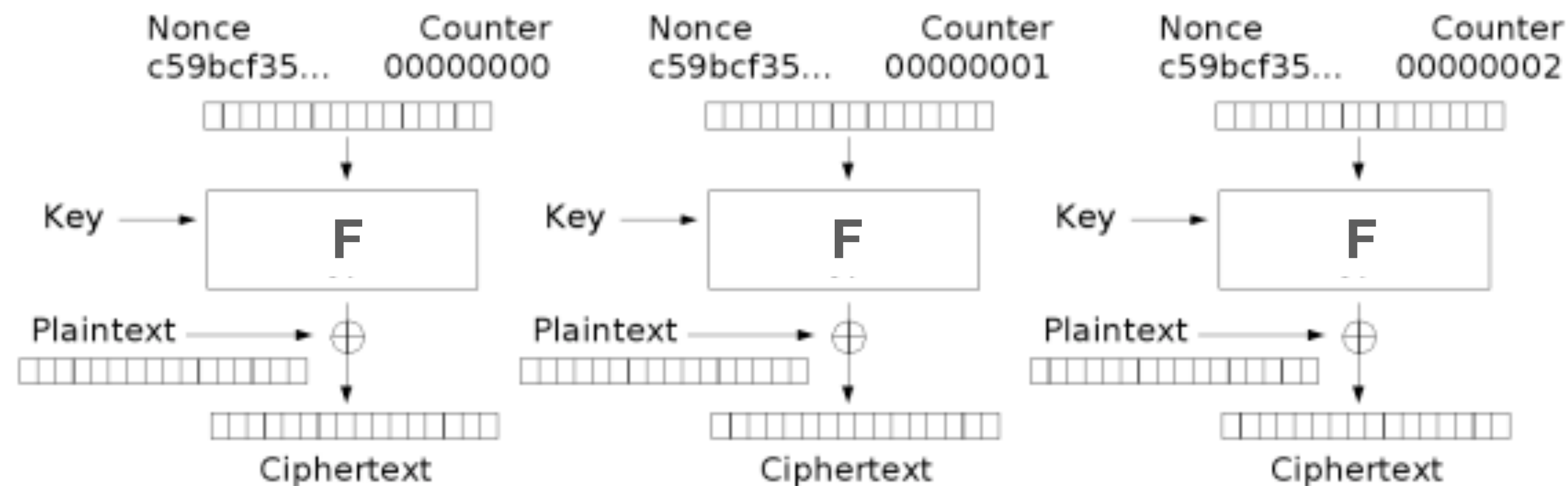
- Similar example: MS Word 2003
 - (they used RC4, but same problem)

CTR with other functions

- We've been assuming a cipher that is invertible (a permutation/block cipher)
- **Observation:** CTR never uses the “Decipher” (invert) algorithm of the cipher
 - So what if we don't use a block cipher at all?

CTR with other functions

- We've been assuming a cipher that is invertible (a permutation/block cipher)
- **Observation:** CTR never uses the “Decipher” (invert) algorithm of the cipher
- So what if we don't use a block cipher at all?



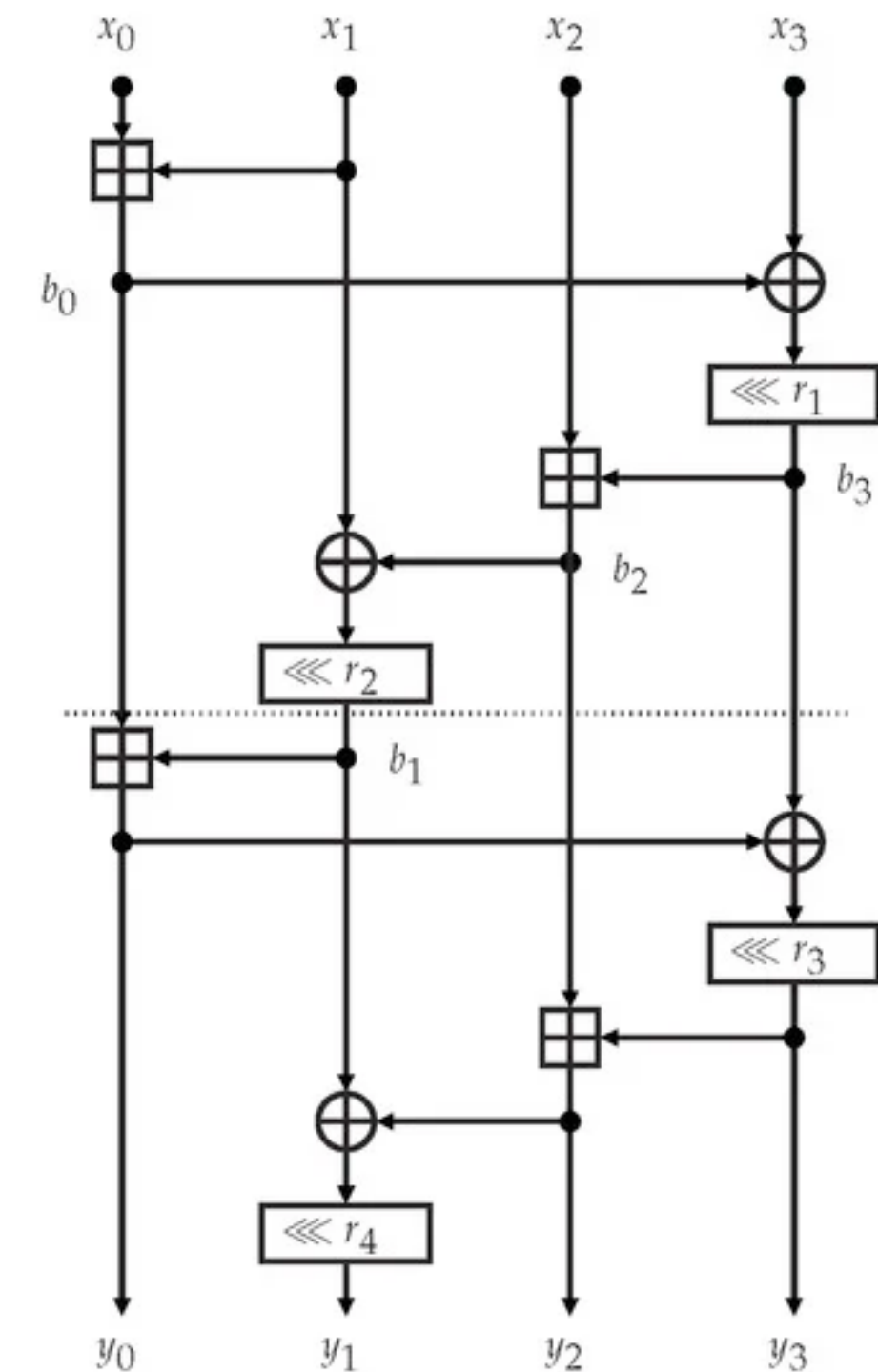
Counter (CTR) mode encryption

ChaCha20

- An example is the ChaCha20 stream cipher
 - Invented by Daniel J. Bernstein (DJB)
- ChaCha20 has one function:

$$F : \underbrace{\{0, 1\}^{256}}_{\text{key}} \times \underbrace{\{0, 1\}^{512}}_{\text{counter (nonce)}} \rightarrow \underbrace{\{0, 1\}^{512}}_{\text{keystream}}$$

- Note: ChaCha is not a permutation! It is not invertible and may output the same value on two different inputs.
- ChaCha20 is faster in software than AES, unless you have hardware support



ChaCha20

```
#define ROTL(a,b) (((a) << (b)) | ((a) >> (32 - (b))))
#define QR(a, b, c, d) ( \
    a += b, d ^= a, d = ROTL(d,16), \
    c += d, b ^= c, b = ROTL(b,12), \
    a += b, d ^= a, d = ROTL(d, 8), \
    c += d, b ^= c, b = ROTL(b, 7))
#define ROUNDS 20

void chacha_block(uint32_t out[16], uint32_t const in[16])
{
    int i;
    uint32_t x[16];

    for (i = 0; i < 16; ++i)
        x[i] = in[i];
    // 10 loops x 2 rounds/loop = 20 rounds
    for (i = 0; i < ROUNDS; i += 2) {
        // Odd round
        QR(x[0], x[4], x[ 8], x[12]); // column 0
        QR(x[1], x[5], x[ 9], x[13]); // column 1
        QR(x[2], x[6], x[10], x[14]); // column 2
        QR(x[3], x[7], x[11], x[15]); // column 3
        // Even round
        QR(x[0], x[5], x[10], x[15]); // diagonal 1 (main diagonal)
        QR(x[1], x[6], x[11], x[12]); // diagonal 2
        QR(x[2], x[7], x[ 8], x[13]); // diagonal 3
        QR(x[3], x[4], x[ 9], x[14]); // diagonal 4
    }
    for (i = 0; i < 16; ++i)
        out[i] = x[i] + in[i];
}
```

Point of order

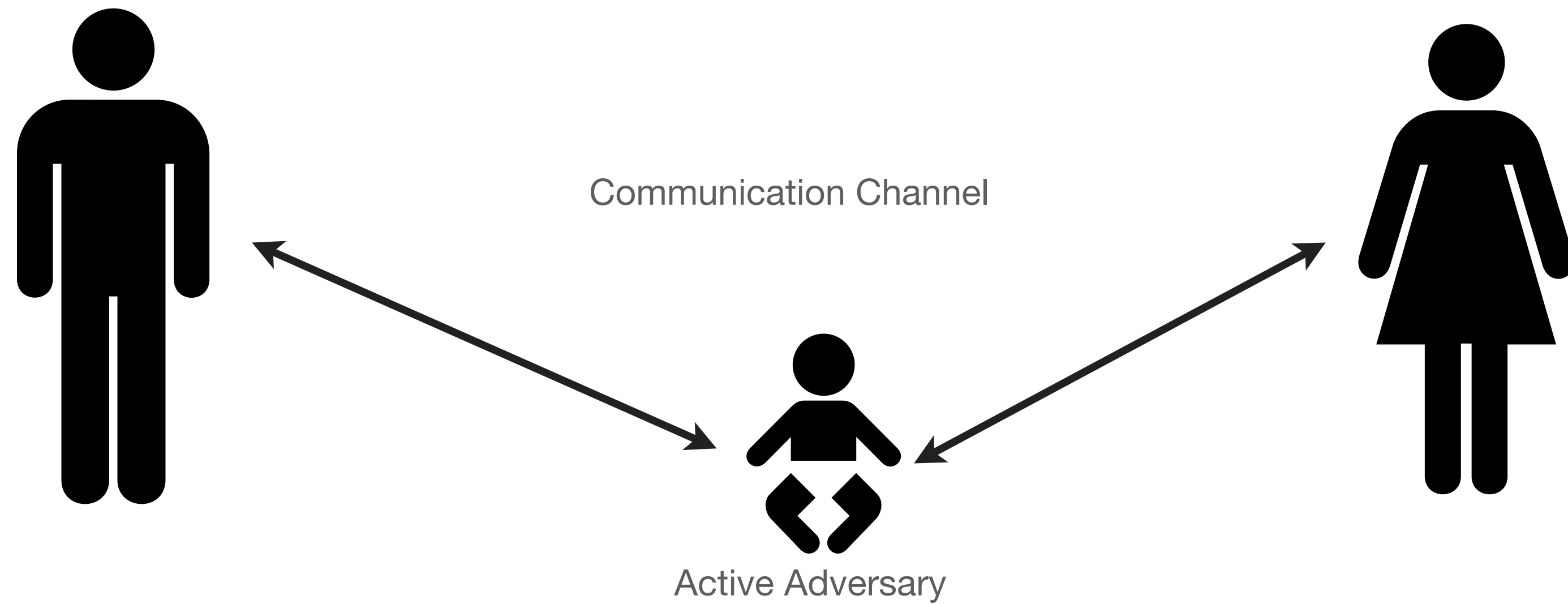
- Proofs of security:
 - We don't know how to prove that DES or AES are secure block ciphers
 - But if we assume that the block ciphers are secure PRPs then:
- We can prove that CBC & CTR & OFB & CFB etc. are secure encryption modes.

<http://www.cs.ucdavis.edu/~rogaway/papers/sym-enc-abstract.html>

Malleability

- The ability to modify a ciphertext
 - Such that the plaintext is meaningfully altered
 - CTR Mode (bad)
 - CBC Mode (somewhat bad)

Authenticated Encryption



MACs

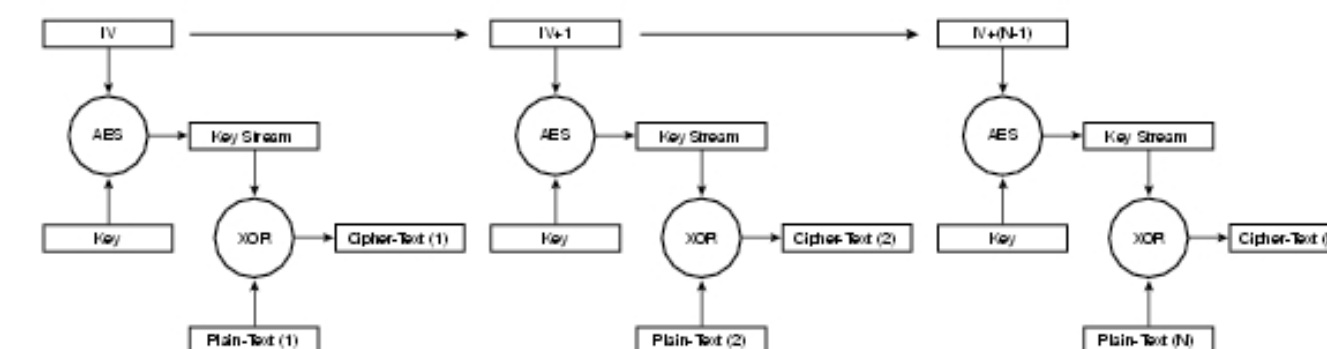
- Symmetric-key primitive
 - Given a key and a message, compute a “tag”
 - Tag can be verified using the same key
 - Any changes to the message detectable
- To prevent malleability:
 - Encrypt then MAC
 - Under separate keys

MACs

- Definitions of Security
 - Existential Unforgeability under Chosen Message Attack (EU-CMA)
- Examples:
 - HMAC (based on hash functions)
 - CMAC/CBC-MAC (block ciphers)

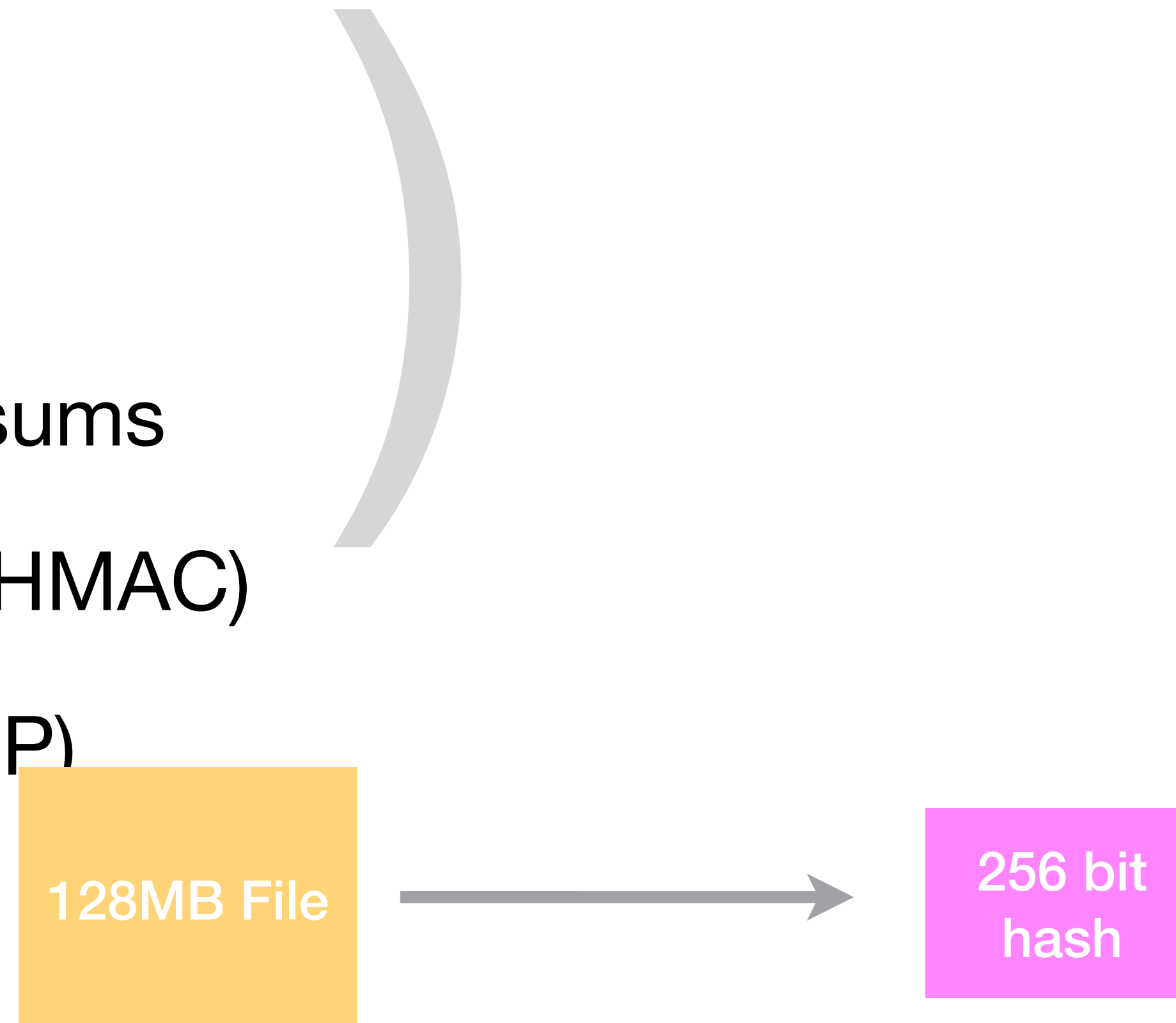
Authenticated Encryption

- Two ways to get there:
 - Generic composition
Encrypt (e.g., CBC mode) then MAC
- two different keys, multiple primitives
 - Authenticated mode of operation
- Integrates both encryption & authentication
- Single key, typically uses only one primitive (e.g., block cipher)
- Ex: CCM, OCB, GCM modes



Hash Functions

- Convert variable-length string to small “tag”
 - Hash tables
 - Signatures
 - Software checksums
 - MAC functions (HMAC)
 - Encryption (OAEP)



Security of hash functions?

Security of hash functions?

- Collision-resistance
- Preimage resistance
- Second preimage resistance

Let's talk about collisions